

Workflow Temporal

Código completo do *workflow* de *deploy* do MAAS, localizado em `src/maastemporalworker/workflow/deploy.py`.

```
import asyncio
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Any

from sqlalchemy import Result, select
from sqlalchemy.ext.asyncio import AsyncConnection
import structlog
from temporalio import workflow
from temporalio.common import RetryPolicy
from temporalio.exceptions import (
    CancelledError, ChildWorkflowError, TimeoutError,
)

from maascommon.enums.node import NodeStatus
from maascommon.enums.power import PowerState
from maascommon.workflows.deploy import (
    DEPLOY_MANY_WORKFLOW_NAME, DEPLOY_WORKFLOW_NAME,
    DeployManyParam, DeployParam, DeployResult,
)
from maascommon.workflows.power import (
    PowerCycleParam, PowerOnParam, PowerParam, PowerQueryParam,
)
from maasservicelayer.builders.nodes import NodeBuilder
from maasservicelayer.db.tables import (
    BlockDeviceTable, InterfaceIPAddressTable, InterfaceTable,
    NodeTable, PhysicalBlockDeviceTable, StaticIPAddressTable,
    VirtualBlockDeviceTable,
)
from maastemporalworker.workflow.activity import ActivityBase
from maastemporalworker.workflow.power import (
    POWER_ACTION_ACTIVITY_TIMEOUT, POWER_CYCLE_ACTIVITY_NAME,
    POWER_ON_ACTIVITY_NAME, POWER_QUERY_ACTIVITY_NAME,
    SET_POWER_STATE_ACTIVITY_NAME, SetPowerStateParam,
)
from maastemporalworker.workflow.utils import (
    activity_defn_with_context, workflow_run_with_context,
)

logger = structlog.get_logger()
DEFAULT_DEPLOY_ACTIVITY_TIMEOUT = timedelta(seconds=30)
DEFAULT_DEPLOY_RETRY_TIMEOUT = timedelta(seconds=60)

GET_BOOT_ORDER_ACTIVITY_NAME = "get-boot-order"
SET_NODE_STATUS_ACTIVITY_NAME = "set-node-status"
MARK_NODE_FAILED_ACTIVITY_NAME = "mark-node-failed"
SET_BOOT_ORDER_ACTIVITY_NAME = "set-boot-order"

class InvalidMachineStateException(Exception):
    pass
```

```
@dataclass
class SetNodeStatusParam:
    system_id: str
    status: NodeStatus
```

```
@dataclass
class MarkNodeFailedParam:
    system_id: str
    message: str
```

```
@dataclass
class GetBootOrderParam:
    system_id: str
    netboot: bool
```

```
@dataclass
class GetBootOrderResult:
    system_id: str
    order: list[dict[str, Any]]
```

```
@dataclass
class SetBootOrderParam:
    system_id: str
    power_params: PowerParam
    order: list[dict[str, Any]]
```

```
class DeployActivity(ActivityBase):
    @activity_defn_with_context(name=SET_NODE_STATUS_ACTIVITY_NAME)
    async def set_node_status(self, params: SetNodeStatusParam) -> None:
        async with self.start_transaction() as services:
            builder = NodeBuilder(status=params.status)
            await services.nodes.update_by_system_id(
                system_id=params.system_id, builder=builder
            )

    @activity_defn_with_context(name=MARK_NODE_FAILED_ACTIVITY_NAME)
    async def set_node_failed(self, params: MarkNodeFailedParam) -> None:
        async with self.start_transaction() as services:
            await services.nodes.mark_failed(
                system_id=params.system_id, message=params.message
            )

    @activity_defn_with_context(name=GET_BOOT_ORDER_ACTIVITY_NAME)
    async def get_boot_order(
        self, params: GetBootOrderParam
    ) -> GetBootOrderResult:
        async with self.start_transaction() as tx:
            # Consulta boot interface e boot disk, constroi ordem
            boot_iface = await self._get_boot_iface(tx, params.system_id)
            boot_disk = await self._get_boot_disk(tx, params.system_id)
```

```

# Monta lista ordenada: se netboot=True, interfaces primeiro;
# se False, discos primeiro
order = []
if params.netboot:
    order = ifaces + block_devs
else:
    order = block_devs + ifaces
return GetBootOrderResult(
    system_id=params.system_id,
    order=[self._stringify_datetime_fields(dev) for dev in order],
)

@workflow.defn(name=DEPLOY_MANY_WORKFLOW_NAME, sandboxed=False)
class DeployManyWorkflow:
    """Workflow pai: gerencia deploy de N máquinas."""

    async def _set_status(self, system_id, status):
        await workflow.execute_activity(
            SET_NODE_STATUS_ACTIVITY_NAME,
            SetNodeStatusParam(system_id=system_id, status=status),
            task_queue="region",
            start_to_close_timeout=DEFAULT_DEPLOY_ACTIVITY_TIMEOUT,
            retry_policy=RetryPolicy(
                maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
            ),
        )

    async def _mark_failed(self, system_id, msg):
        await workflow.execute_activity(
            MARK_NODE_FAILED_ACTIVITY_NAME,
            MarkNodeFailedParam(system_id=system_id, message=msg),
            task_queue="region",
            start_to_close_timeout=DEFAULT_DEPLOY_ACTIVITY_TIMEOUT,
            retry_policy=RetryPolicy(
                maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
            ),
        )

@workflow_run_with_context
async def run(self, params: DeployManyParam) -> None:
    pending: set[workflow.ChildWorkflowHandle] = set()
    for param in params.params:
        wf = await workflow.start_child_workflow(
            DEPLOY_WORKFLOW_NAME, param,
            id=f"deploy:{param.system_id}",
            task_queue=param.task_queue,
            retry_policy=RetryPolicy(
                maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
            ),
            execution_timeout=timedelta(minutes=param.timeout),
        )
        pending.add(wf)

    while pending:
        done, pending = await asyncio.wait(

```

```

        pending, return_when=asyncio.FIRST_COMPLETED
    )
    for t in done:
        system_id = t.id.removeprefix("deploy:")
        if e := t.exception():
            msg = ""
            if isinstance(e, (CancelledError, asyncio.CancelledError)):
                continue
            elif isinstance(e, ChildWorkflowError):
                if isinstance(e.cause, CancelledError):
                    continue
                elif isinstance(e.cause, TimeoutError):
                    msg = "time-out during deployment"
                else:
                    msg = str(e.cause)
            else:
                msg = str(e)
            await self._mark_failed(system_id, msg)
        else:
            if t.result()["success"]:
                await self._set_status(
                    system_id, NodeStatus.DEPLOYED
                )

```

```

@workflow.defn(name=DEPLOY_WORKFLOW_NAME, sandboxed=False)

```

```

class DeployWorkflow:

```

```

    """Workflow filho: deploy de uma única máquina."""

```

```

    def __init__(self) -> None:
        self._has_netbooted = False
        self._deployed_os_ready = False

```

```

    @workflow.signal(name="netboot-finished")
    async def netboot_signal(self, *args: list[Any]) -> None:
        self._has_netbooted = True

```

```

    @workflow.signal(name="deployed-os-ready")
    async def deployed_os_signal(self, *args: list[Any]) -> None:
        self._deployed_os_ready = True

```

```

    async def _start_deployment(self, params: DeployParam) -> None:
        result = await workflow.execute_activity(
            POWER_QUERY_ACTIVITY_NAME,
            PowerQueryParam(**params.power_params.__dict__),
            task_queue=params.power_params.task_queue,
            start_to_close_timeout=POWER_ACTION_ACTIVITY_TIMEOUT,
            retry_policy=RetryPolicy(
                maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
            ),
        )

```

```

    if result["state"] == PowerState.ON:
        new_result = await workflow.execute_activity(
            POWER_CYCLE_ACTIVITY_NAME,
            PowerCycleParam(**params.power_params.__dict__),

```

```

        task_queue=params.power_params.task_queue,
        start_to_close_timeout=POWER_ACTION_ACTIVITY_TIMEOUT,
        retry_policy=RetryPolicy(
            maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
        ),
    )
else:
    new_result = await workflow.execute_activity(
        POWER_ON_ACTIVITY_NAME,
        PowerOnParam(**params.power_params.__dict__),
        task_queue=params.power_params.task_queue,
        start_to_close_timeout=POWER_ACTION_ACTIVITY_TIMEOUT,
        retry_policy=RetryPolicy(
            maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
        ),
    )
if new_result["state"] != result["state"]:
    await workflow.execute_activity(
        SET_POWER_STATE_ACTIVITY_NAME,
        SetPowerStateParam(
            system_id=params.power_params.system_id,
            state=PowerState(new_result["state"]),
        ),
        task_queue="region",
        start_to_close_timeout=DEFAULT_DEPLOY_ACTIVITY_TIMEOUT,
        retry_policy=RetryPolicy(
            maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
        ),
    )
)

async def _set_boot_order(self, params: DeployParam) -> None:
    boot_order = await workflow.execute_activity(
        GET_BOOT_ORDER_ACTIVITY_NAME,
        GetBootOrderParam(
            system_id=params.system_id, netboot=False,
        ),
        task_queue="region",
        start_to_close_timeout=DEFAULT_DEPLOY_ACTIVITY_TIMEOUT,
        retry_policy=RetryPolicy(
            maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
        ),
    )
    if boot_order:
        await workflow.execute_activity(
            SET_BOOT_ORDER_ACTIVITY_NAME,
            SetBootOrderParam(
                system_id=params.system_id,
                power_params=params.power_params,
                order=boot_order["order"],
            ),
            task_queue=params.power_params.task_queue,
            start_to_close_timeout=POWER_ACTION_ACTIVITY_TIMEOUT,
            retry_policy=RetryPolicy(
                maximum_interval=DEFAULT_DEPLOY_RETRY_TIMEOUT
            ),
        )
    )

```

```
    else:
        raise InvalidMachineStateException("no boot order found")

@workflow_run_with_context
async def run(self, params: DeployParam) -> DeployResult:
    await self._start_deployment(params)

    if not params.ephemeral_deploy:
        await workflow.wait_condition(lambda: self._has_netbooted)
        if params.can_set_boot_order:
            await self._set_boot_order(params)

    await workflow.wait_condition(lambda: self._deployed_os_ready)
    return DeployResult(system_id=params.system_id, success=True)
```